Description:

This project is a real-time interactive video processing application implemented in C++ using OpenCV. The app captures live video from a webcam, then based on user keyboard input, applies to the video several filters and special effects. User can press the same key to toggle on and off a specific effect.

My pipieline: capture → view mode → stackable effects → rotation → display/record

View Modes (mutually exclusive):

'o': Original view

'g': Grayscale (OpenCV built-in)

'h': Custom grayscale implementation

'x': Sobel X edge detection

'y': Sobel Y edge detection

'm': Gradient magnitude

'd': Depth map visualization

'D': Depth-grayscale blend effect

'z': Depth fog effect

'e': Emboss effect

'c': Face color pop

Stackable Effects:

'b': Custom 5×5 blur

'F': Horizontal flip

'v': Color inversion

'p': Sepia tone

'i': Color quantization

'f': Face detection with bounding boxes

Additional Features:

'r': Rotate 90° clockwise (persistent)

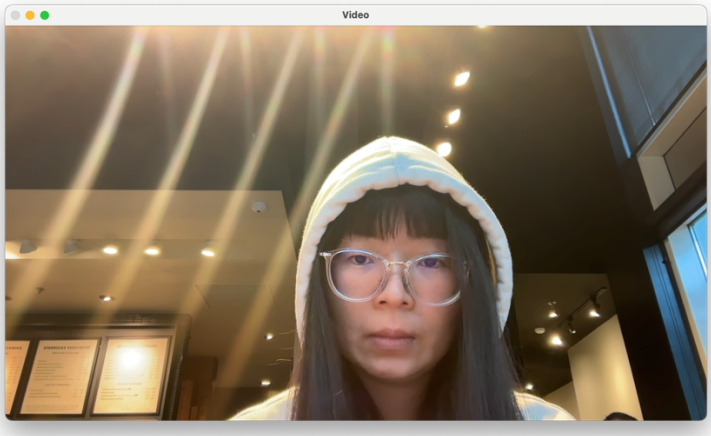
's': Save current frame as PNG screenshot

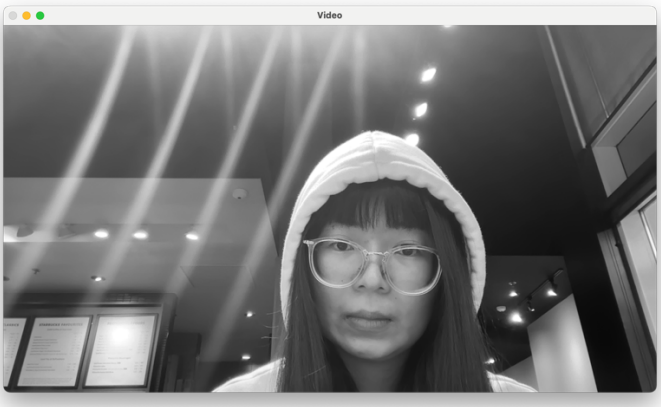
'V': Toggle video recording (MP4 output)

't': Print frame type information to console

'q': Quit application

**Task 3 — Required Image 1:** show the original and cvtColor version of the greyscale image in your report.




I store the last key selection in a mode variable (MODE_COLOR, MODE_GRAY, MODE_BLUR, MODE_FLIP, MODE_INVERT). Each captured frame is processed according to the current mode so the effect persists across frames.

how each color channel is weighted in the conversion:

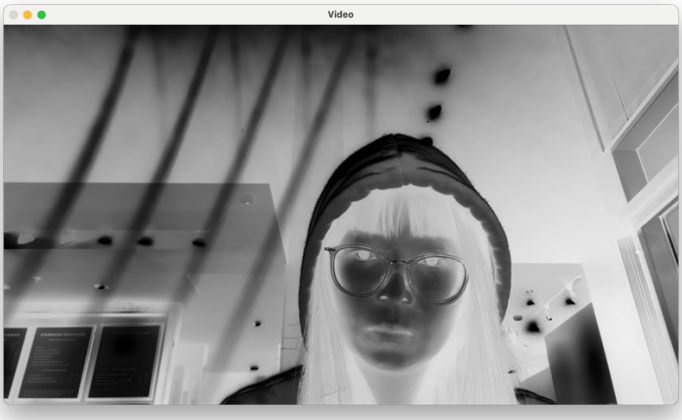
$$\text{RGB[A] to Gray:} \quad Y \leftarrow 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

I checked Wikipedia for this equation and learned that this RGB-to-grayscale conversion is based on the Rec. 601 luma standard.

The green channel contributes the most to perceived brightness because the human eye is most sensitive to green wavelengths, while the blue channel contributes the least because human vision is less sensitive to blue light in terms of brightness perception.

**Task 4 — Required image 2:** show your customized greyscale image in your report, explain how you decided to generate your greyscale version, and highlight the differences with the default greyscale image.

My customized greyscale image:




how I decided to generate my greyscale version:

Invert the red channel value: R->255-R, and use this value for each of the three channels so that it becomes a greyscale image.

the differences with the default greyscale image:

| | default | My custom |
|---|---|---|
| Number of channels | 1 | 3 |
| How the value of Luma component Y ′is calculated | 0.299R+0.587G+0.114B | 255-R |
| Whether it aligns with human brightness perception | Yes | No |

**Task 5 — Required image 3:** show your sepia tone filter in your report and explain how you ensured using the original RGB values in the computation.


how I ensured using the original RGB values in the computation:
I avoided modifying the source image in place, instead, I created a destination cv::Mat as a buffer, read the value from the source, do the computation and then write results to the destination. Then I made a deep copy of des: `sepiaImg.copyTo(display);`

**Task 6 — Required Image 4:** show the original and the blurred image in your report along with the timing information.

The output of timing:

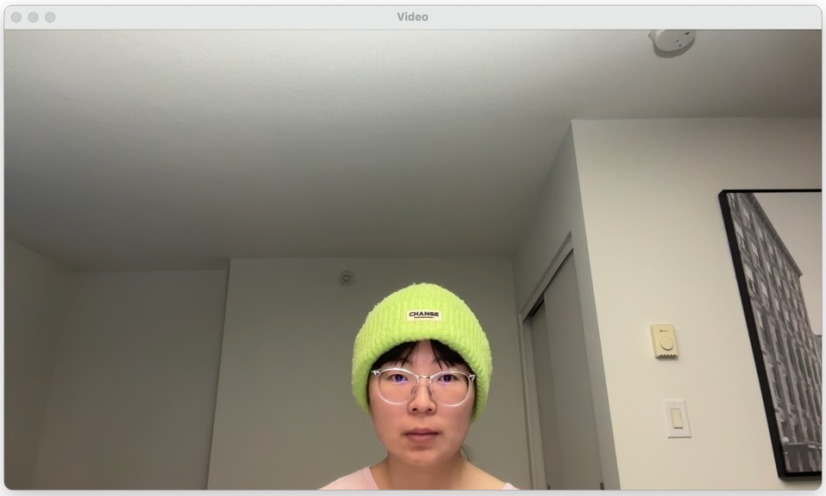
Time per image (1): 0.1741 seconds

Time per image (2): 0.0264 seconds

Terminating

blur5x5_2 is faster because it uses a separable filter, use one horizontal $1 \times 5$ filter and one vertical $1 \times 5$ filter to replace the $5 \times 5$. The number of operations per pixel dropped from 25 to 10.

Another reason is in blur5x5_2 I used row pointers Mat::ptr<> instead of Mat::at<>, this is more cache friendly.

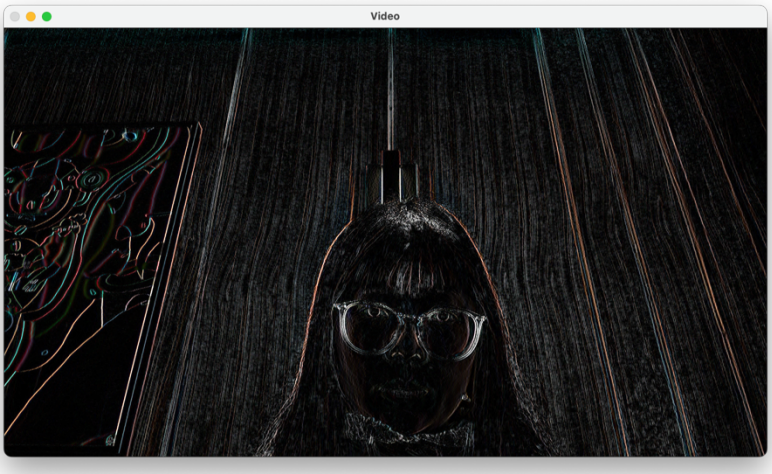




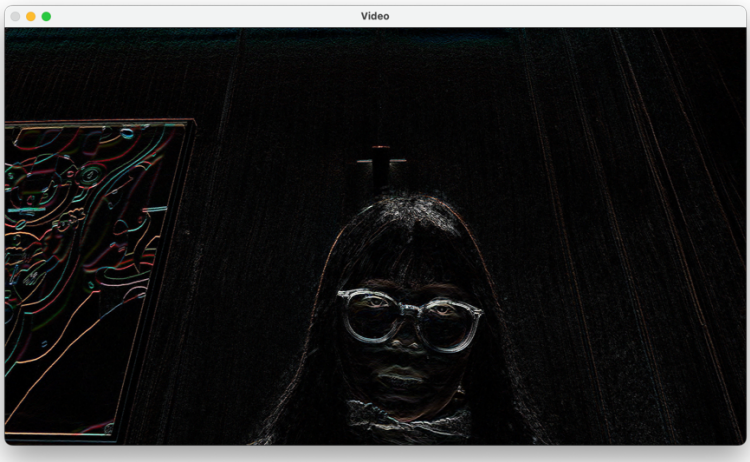
**Task 8 — Required Images 5:** show the original, X-Sobel, Y-Sobel, and the gradient magnitude image in your report.

The original:

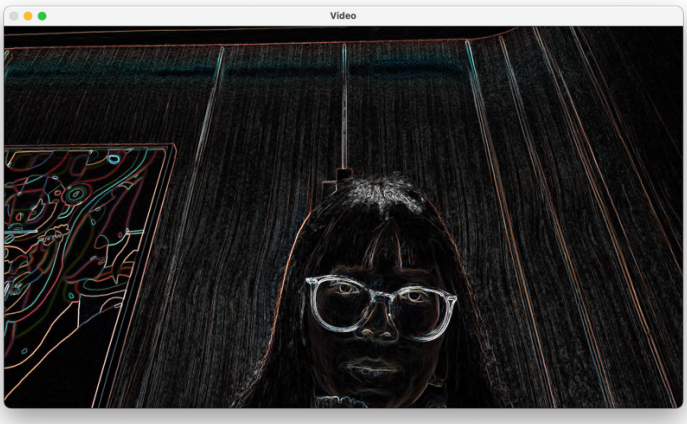
Sodel X:




Sodel Y:




the gradient magnitude:



This group of images show edge detection.

Sobel X computes the horizontal intensity gradient of the entire image. Large horizontal intensity gradient corresponds with vertical edges, where intensity changes rapidly from left to right. The gradient values are visualized by taking their absolute values and scaling them to an 8-bit range.

Similarly, Sobel Y computes the vertical intensity gradient of the entire image. Large horizontal vertical gradient corresponds with horizontal edges.

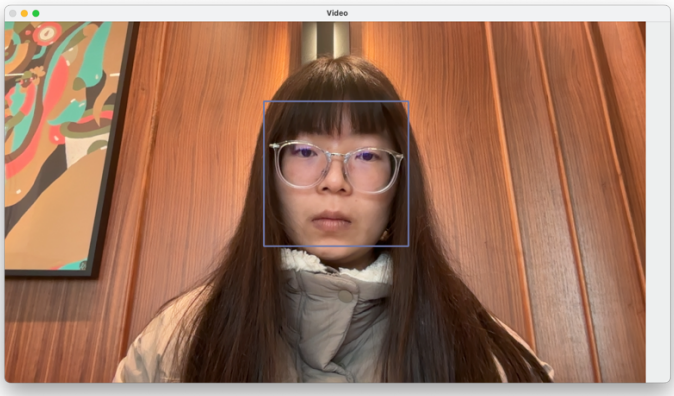
The gradient magnitude image combines both directional gradients using $\sqrt{(Sx^2 + Sy^2)}$, as a result we get a direction-independent edge map.

Task 9:

Required Image 6: show the original and the blurred/quantized image in your report.

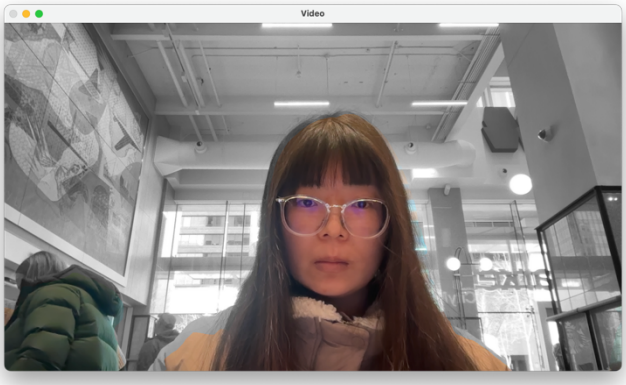

This image shows the combined blur and color quantization effect. First we apply a custom 5×5 blur filter to smooth the image, reduce the high frequency noise and also the local variations. Then the quantization process is we map each color channel to 10 buckets. So, there are at most $10^3 = 1000$ colors in the image. In other words, we have significantly reduced the color resolution.

**Task 10 — Required Image 7:** show a face being detected in your video stream.




I used the given code and realized the real-time face detection which uses OpenCV's pre-trained Haar cascade classifier. First convert the frame to grayscale, and then call detectFaces(), finally draw boxes around detected faces.

**Task 11 — Required Images 8-9:** show a depth image from your video stream (to prove it's working) and an image of your creative filter that makes use of depth values.

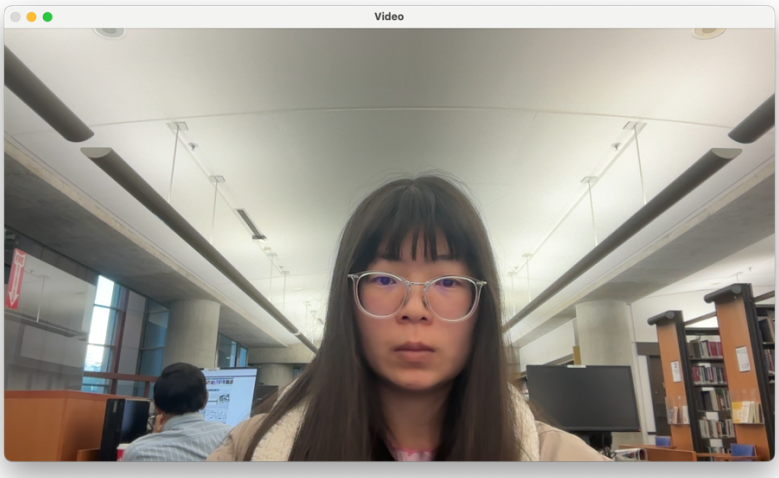




Depth Anything V2 neural network performs monocular depth estimation. Image 8 is the output depth map. It is represented as an 8-bit grayscale image, where lighter values correspond to closer objects and darker values correspond to farther objects.

To strike a balance between speed and depth quality, I let the network run at 40% input resolution and update every three frames.

Image 9 is the frame after applying my filter. My filter is very simple, turn pixels beyond a depth threshold into gray using this equation: $\boldsymbol{Y = 0.299\,R + 0.587\,G + 0.114\,B}$.

**Task 12 — Required Images 10-12:** show the original and the modified images in your report.



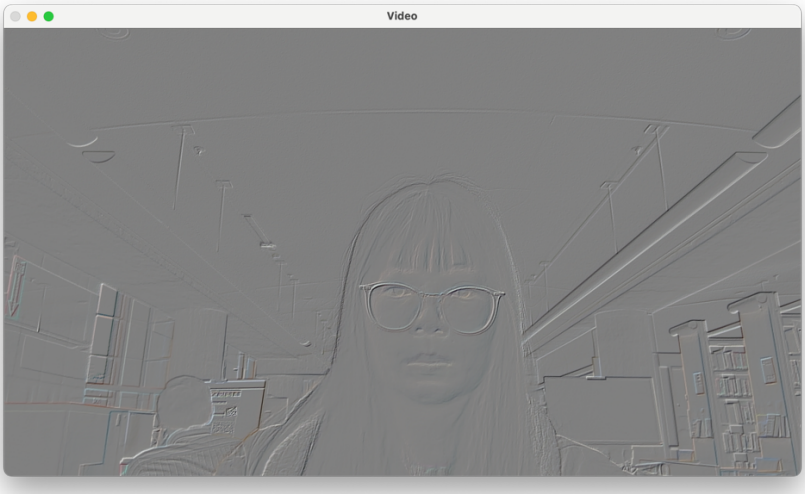




Image 10:

First compute image gradients, both Sobel X and Y gradients. Then we assume a light direction, for example vector (0.707, 0.707), then project the gradient vector onto a fixed light direction via a dot product: $a \cdot b = |a||b|cos\theta$

This produces a shading effect where edges aligned with the light direction appear brighter, while opposing edges appear darker, simulating a relief surface.

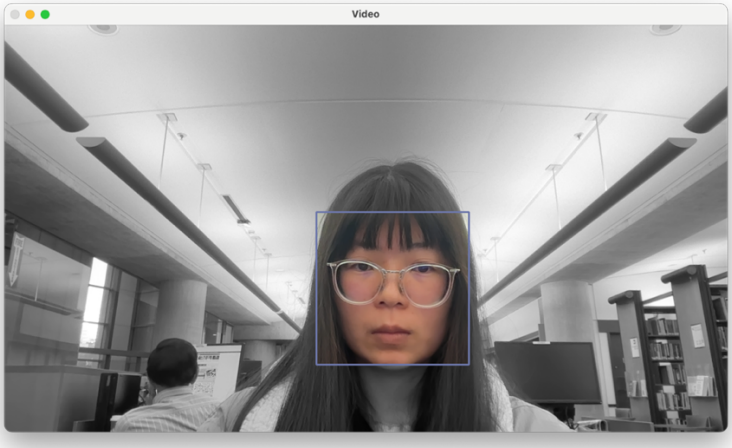
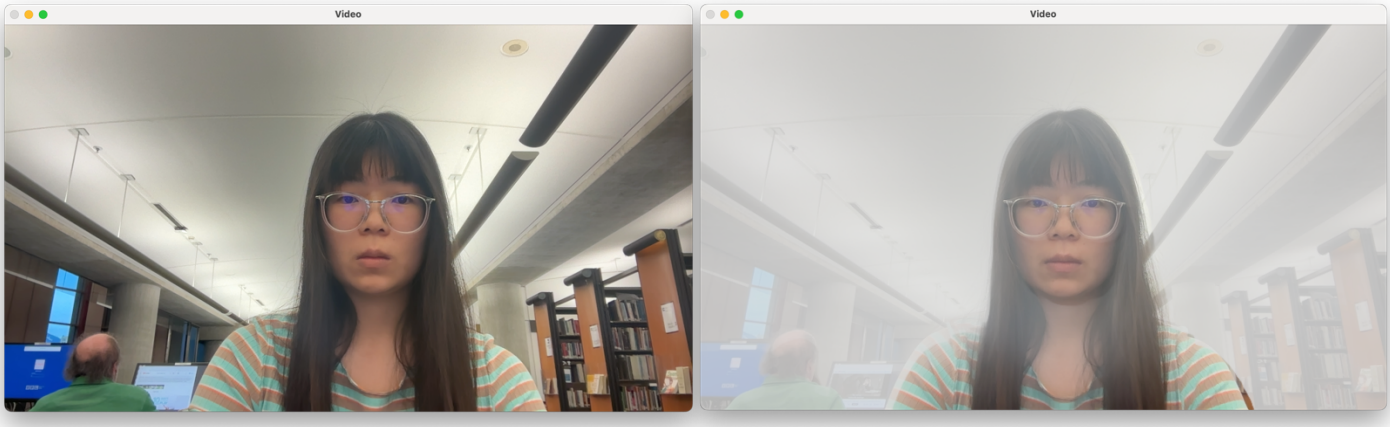



Image 11:

The face color pop effect combines face detection with selective color manipulation. The entire image is first converted to grayscale, then detected face regions are copied back in full color. Before copying back, each detected face bounding box is intersected with the valid image region, preventing out-of-bounds memory access.

Image 12:




This depth-based fog effect blends the original image with a fog color using an exponential falloff model. The intended effect is that the fog intensity increases with distance. Because Depth Anything V2 network gives inverse depth, meaning larger values correspond to closer regions, to ensure that fog increases with distance, the depth values are inverted using 1 - d before applying the exponential fog model.

The fog density parameter controls how quickly the fog gets stronger in terms of distance.

**Extensions — video recording**

Sample video

This functionality allows users to save short video sequences demonstrating the implemented effects directly from the live stream.

If user press 'V', video recording starts, and it will write each processed frame (after view mode, effects, and rotation) to a MP4 under the output folder. VideoWriter needs a constant frame size and the type to be CV_8UC3. Because user might have rotated the image for 90° or 270° before recording, it's necessary that frames be resized before writing.

**A short reflection of what you learned:**

During this assignment, I learned a lot about the differences between C and C++. For example, the difference between references and pointers in C++, and how they affect function design and readability. I also became more aware that C++ allows both a C-style procedural approach and an object-oriented style, and I started to understand when each is appropriate.

I initially tried to work with Makefiles and used AI tools to carefully go through the Makefile given by Professor Maxwell line by line, also managed to adapt it to run on my own machine. Although I am not yet fully comfortable writing Makefiles from scratch, I can now read and modify them with much more confidence. Eventually, I chose to use an IDE since I still found it more convenient, but the experience helped me understand the build process more clearly instead of treating it as a black box.

From an image processing perspective, this assignment helped demystify common operations such as filtering and edge detection. I realized even those techniques that seemed complex at first are actually built from simple operations. This makes me realize how important it is to lay a solid foundation, no limited to C++, in every sense.

Finally, I learned the value of reading official documentation. Previously, I relied heavily on online forums or AI-generated explanations, but this assignment encouraged me to consult primary sources such as the OpenCV documentation and tutorials. I found that the documentation was more approachable than I expected and often clearer and more reliable than secondary sources.

**Acknowledgement of any materials or people you consulted for the assignment.**

I started from learning from the sample code Professor Bruce provided. All the code files are clear and well documented.

I consulted the official OpenCV documentation and OpenCV tutorials for understanding built-in filtering functions, video input/output and advanced image processing like filtering and edge detection. I also consulted the CMake documentation for building configuration.

I used ChatGPT to explain a lot of things for me, some are image processing related, for example, in task 9, we need to implement a function that blurs and quantizes a color image. ChatGPT helps me understand why we need to blur first. I also used ChatGPT to teach me a lot about C++ since before this course I had zero experience in C++. For example, it taught me how to organize my files, how to edit my CMakeLists.txt for me to be able to run more than one apps in the same project. Another example is I used ChatGPT to better understand the sample code given by Professor Maxwell. Through this project I have become comfortable working with this language, and I really appreciate Professor Maxwell for teaching this course in C++. Now I have another new tool in my toolkit thanks to this course.

I used VS code AI Agent to write README.md and documentation.